



# WEB SECURITY



## AUTHENTICATION

**Lê Thị Minh Châu**  
**Faculty Of Information Technology**  
**HCMC University Of Technology And Education**

# Outline



2

1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism

# Outline



3

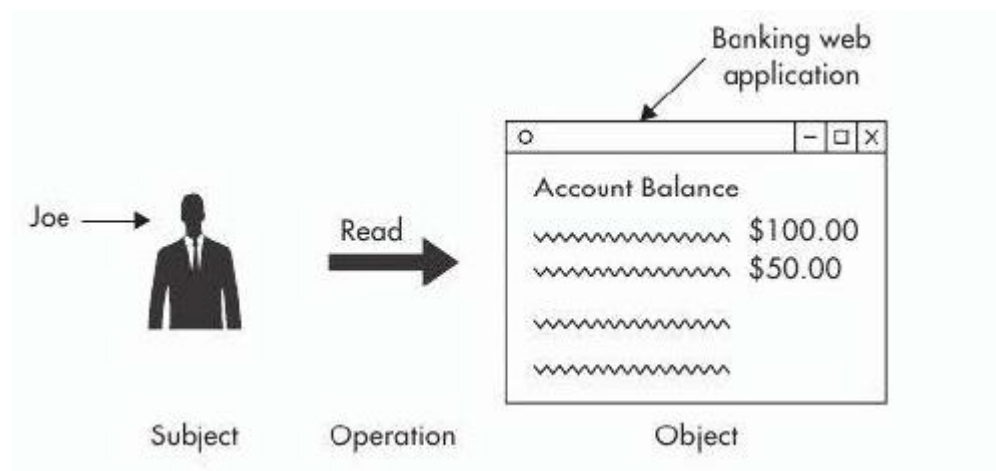
1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism



# Access Control Overview

4

- **Access control system**: mechanism that regulates **access** to data or functionality by determine whether a subject is permitted to perform an operation on a target object.



A simple model Access Control

# Access Control Overview (cont.)



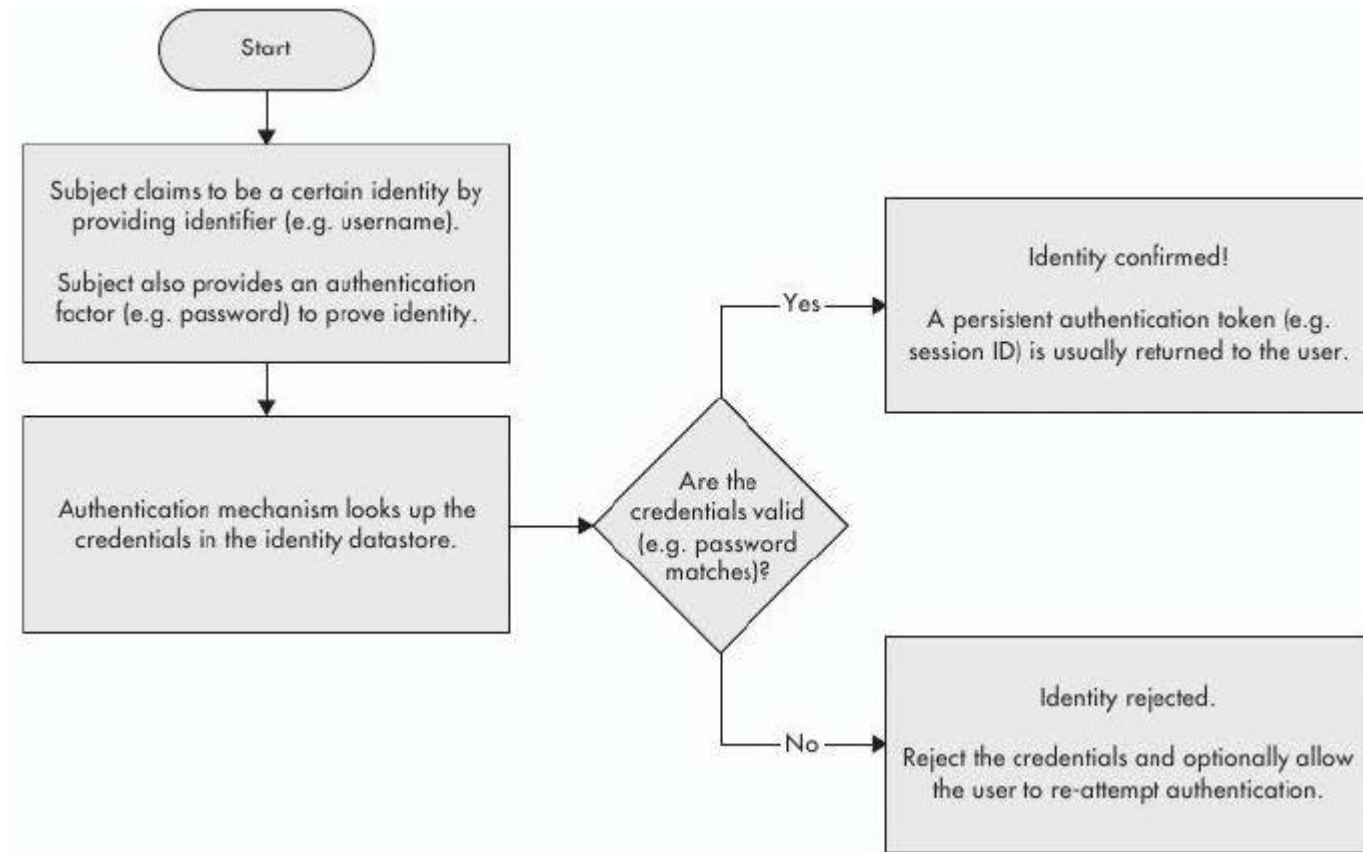
5

- Access Control Mechanism relies on two related process:
  - ▣ **Authentication**: process in which a subject proves that they are who they claim to be.
  - ▣ **Authorization**: process of determining whether the validated identity has the rights to do what they want to do.

# Access Control Overview (cont.)



6



Access Control Process

# Outline



7

1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism

# Authentication Fundamentals



8

- Authentication has two steps:
  - ▣ **Identification**: claims to be a certain person
  - ▣ **Confirmation**: allows a subject to prove that claim



# Authentication Fundamentals (cont)



9

- Proving the Identity: 3 classes
  - Something you *know*: *password, PIN, pass phrase*
  - Something you *have*: *digital certificate, smart card, security token (RSA SecurID)*
  - Something you *are*: *fingerprint, retinal pattern, hand geometry, topography of your face*

# Authentication Fundamentals (cont)



10



RSA SecurID Security Token

# Authentication Fundamentals (cont)



11



Fingerprint Scanner

# Authentication Fundamentals (cont)



12

- **Two – Factor Authentication:** authentication of someone's identity is performed using factors from two of the three categories (*know, have, are*)
  - ▣ Ex: ATM (card, PIN)
- **Three – Factor Authentication:**

# Outline



13

1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism

# Web Application Authentication



14

- Password – Based Authentication Systems
  - ▣ Username & password
  - ▣ The HTTP specification provides two built-in authentication mechanisms:
    - Basic Access Authentication
    - Digest Access Authentication
  - ▣ Single Sign – on

# Basic Access Authentication



15

1. Request goes out for a file

<http://www.website.cxx/secure/privatefile.htm>

Web Server will respond with the **401**  
*Authorization Required Response Code*

# Basic Authentication (cont.)



16

```
HTTP/1.1 401 Authorization Required
Server: HTTPd/1.0
Date: Sat, 27 Nov 2004 10:18:15 GMT
WWW-Authenticate: Basic realm='' Secure Area''
Content - Type: text/html
Content - Length: 311
c!DOCTYPE HTML PUBLIC '' - //W3C//DTD HTML 4.01
Transitional//EN''
''http://www.w3.org/TR/1999/REC- html401 -
19991224/loose.dtd''>
cHTML>
<HEAD>
cTITLE>Errorc/TITLE>
cMETA HTTP - EQUIV="Content - Type" CONTENT="text/html;
charset-ISO- 8859 - 1">
c/HEAD>
cBODY>cH1>401 Unauthorized.c/H1></BODY>
c/HTML>
```



# Basic Authentication (cont.)



17

2. Browser pop up a dialog box that requests the user enter their credentials.
3. Browser concatenates username with “:” and password, then encode it to 64 – bit and submits via GET request to the Web Server.

VD:      username: stwewie

password: griffin

“stewie: griffin”

→ “c3RJd2lOmdyaWZmaW4=”

# Basic Authentication (cont.)



18



# Basic Authentication (cont.)



19

```
GET /private/index.html HTTP/1.1
```

```
Host: www.website.cxx
```

```
Authorization: Basic c3Rld2110mdyaWZmaW4=
```

4. Server accepts the credentials → return protected resource to the user

Server rejects the credentials → error or another 401 response code may be returned.

# Basic Authentication (cont.)



20

## □ Insecure Transmission

- ▣ Username and password are 64-bit encode, not encrypted.

→ Encrypt credentials or submit over an SSL connection.

# Basic Authentication (cont.)



21

## □ Repeated Exposure

- In the fact, credentials must be submitted with every request for a protected resource.
- Web application does not respond with a session ID, which is used to identify an authenticated session.
- Browser caches the credentials and resubmits them whenever access to a protected resource take place.
- Username & password are exposed over and over with each request to the web server.

# Basic Authentication (cont.)



22

## □ Insecure Storage

- ▣ Browser caches the authentication credentials
- ▣ No way to invalidate a session with the web server since no session is created
- ▣ No way to logout
- ▣ The only way to clear the stored credentials is to close the tab or clear the history

# Digest Access Authentication



23

- Using MD5 to transform the password (along with some other information)
- Using nonce value to make replay attacks more difficult

# Digest Access Authentication (cont.)



24

- **Man – in – the Middle Attack:** it's possible to trick the client into downgrading the security back to Basic Authentication or Older Digest Authentication mechanisms.



# Single Sign – on Authentication



25

- Single Sign – on (SSO): allows a user to **log in to a single interface** and gain access to multiple, independently secure systems.
- With web applications, SSO allows a user to enter their credentials and access multiple web applications that may have been developed and secured independently but tie into or support the SSO system

# Single Sign – on Authentication (cont.)



26

- Google Services:  
Gmail, Google Talk,  
YouTube, and Picasa.
- Thirdparty web  
applications cannot  
integrate with Google  
accounts at this time

One account. All of Google.

Sign in to continue to Gmail

The image shows a Google sign-in interface. At the top is a grey circular placeholder for a profile picture. Below it are two input fields: one labeled 'Email' and one labeled 'Password'. A blue 'Sign in' button is positioned below the password field. At the bottom left, there is a checkbox labeled 'Stay signed in'. At the bottom right, there is a link that says 'Need help?'.

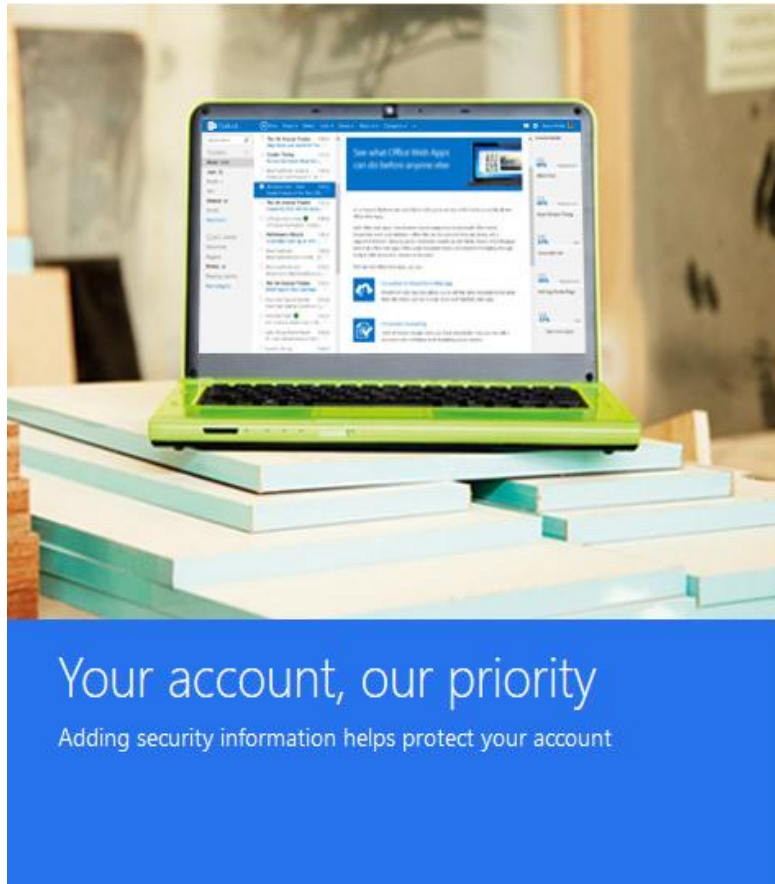
[Create an account](#)

# Single Sign – on Authentication (cont.)



27

- Microsoft's Live ID services: allow third – party website integration



## Sign in

Microsoft account [What's this?](#)

☐ Keep me signed in

[Sign in](#)

[Can't access your account?](#)

[Sign in with a single-use code](#)

Don't have a Microsoft account? [Sign up now](#)

# Single Sign – on Authentication (cont.)



28

- Connect API allows third-party developers to leverage the authentication system in addition to being able to connect with other users as they would on Facebook.
- Sites using Facebook Connect include Digg, CNN, Vimeo, and the Buffington Post

facebook

Email hoặc điện thoại

Mật khẩu

☐ Duy trì đăng nhập

[Đăng nhập](#)

[Quên mật khẩu?](#)

# Single Sign – on Authentication (cont.)



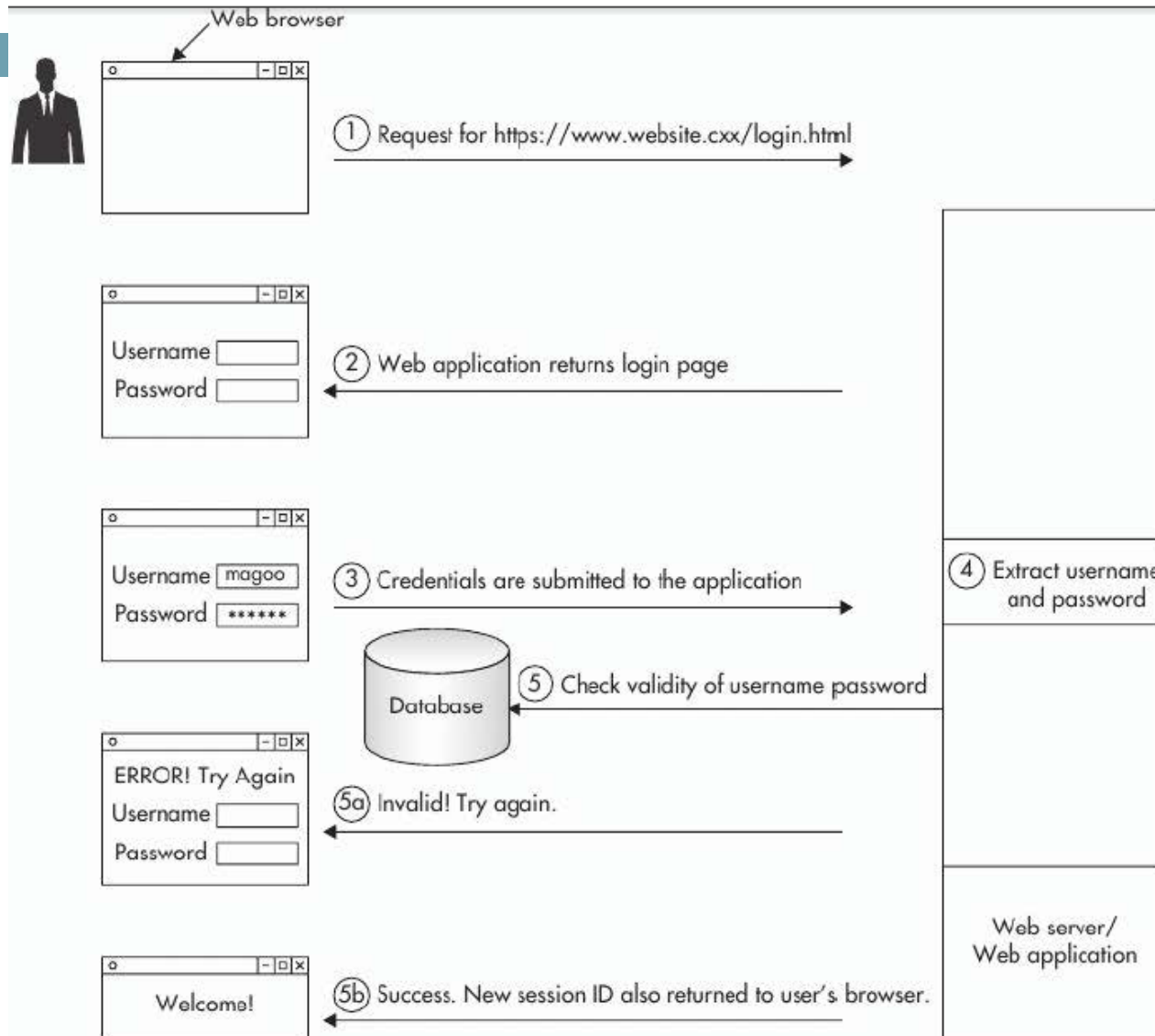
29

- As everyone continues to migrate more toward web-based applications, it will only become more and more difficult to manage credentials for the disparate systems.
- The adoption of SSO will help to solve this problem by reducing the number of credentials that must be remembered.
- ➔ Single set of credentials comes at the risk of a single point of failure:
  - ▣ Your credentials be compromised, more than just your Facebook, Live ID or your profile on any sites that are connected through the same SSO system account being exposed.

# Custom Authentication Systems



30



# Custom Authentication Systems (cont.)



31

5. If the password is matched to the username, the application will establish a session with the user.
  1. Generating a session ID
  2. Session ID is returned by setting a cookie in the HTTP response

# Custom Authentication Systems (cont.)



32

HTTP/1.1 302 Moved Temporarily

Date: Sun, 17 Apr 2011 18:28 :52 GMT

Server: Apache/2.2.15 (Unix) mod\_ssl/2 . 2.15 OpenSSL/1.0.0

Cache-Control: no-cache="Set-Cookie"

Location: <https://www.website.cxx/main.jsp>

**Set-Cookie: JSESSIONID=vPGwNrxDzSshFhd2LJcFqVvYLOs8r3wnzYJ6B5rnJMNwJh**

Obg3PWN!-1251942355;path=/;secure;httponly

X-Powered-By: Servlet/2.5 JSP/2.1

Keep-Alive: timeout=5, max=100

Connection: Keep-Alive

Content-Type: text/html; charset=ISO-8859-1

Content-Length: 437

<html><head><title>302 Moved Temporarily</title></head>



# Custom Authentication Systems (cont.)



33

6. Browser receives and parses the HTTP response, observe the **Set-Cookie** and store value of the session ID.
  - ▣ Submit the session ID value alongside all requests made to the web application
  - ▣ Web application use session ID to authorize each request within the application logic

# Validating Credentials



34

- Validating Credentials is more complex than simply “determining whether or not the supplied password is associated with the supplied user name”.
- Two variables involve in the validation of credentials:
  - ▣ Location of the comparison logic: within application or database.
  - ▣ How the password is stored: plaintext or hashed

# Comparison Logic in the Application with Plaintext Passwords



35

- The application sends a request (SQL or LDAP query) to the back – end database to retrieve the record associated with the username.
  - ▣ Username does not exist: validation process fails.
  - ▣ Otherwise, the application based logic compares the plaintext passwords of the user and the record. If they matched, then the credentials are valid.

# Comparison Logic in the Database with Plaintext Passwords



36

- Creating a SQL query or an LDAP request that asks the back – end system to return any records with matching fields corresponding to the supplied username & password.
- The database compares username & password with the corresponding fields.
  - ▣ If any records are returned, the user provided the legitimate set of credentials (unless SQL injection is used).

# Comparison Logic in the Application with Hashed Password



37

- A request is first made to the back – end datastore to retrieve the record associated with the user – supplied username.
- User – supplied password must be hashed using the same algorithm that was used to hash the stored password
- If two hashed value are equal then authentication succeeds.

# Comparison Logic in the Database with Hashed Password



38

- The back – end system logic attempts to match the supplied username and password (hashed) with a record in the system.
- If a match occurs, then the record is returned and the application assumes that the credentials were valid (unless SQL injection is used).

# Outline



39

1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism

# Securing Password – Based Authentication



40

- Dictionary Attack
- Brute – Force Attack
- Precomputed Distionnary Attack
- Rubber – Hose Attack



# Dictionary Attack



41

- User often selects passwords based on real words that can be found in “dictionaries”.
- With dictionary, the attacker will guess each password until they succeed or the dictionary is exhausted.
- Full dictionaries are not used in online attack because of the time limitation, a targeted dictionary is employed.
- Offline attack usually begin with one of these massive dictionaries to locate any weak passwords before using brute – force attacks for the harder – to – crack passwords.

# Brute – Force Attack



42

- Exhaustive Key Search
- Limits are usually based on a brute – force attack based on length and character set (alphabet, digits, whitespace, special characters)

# Precomputed Dictionary Attack



43

- Dictionary Attack and Brute – Force Attack require to rehash all the dictionary or brute – force values → waste of time.
- Pre – hash all the values in a dictionary and store those values to disk → trading time for disk space.

# Rubber – Hose Attack



44

- Attacker uses any sort of physical coercion to extract the value of the password from an individual.

# Password Complexity



45

- Require minimum password length
- Enforce minimum password complexity:  
Uppercase letters, Lowercase letters,  
Numbers, Symbols, ...
- Rotate passwords: at least 90 days
- Require password uniqueness: don't reuse  
one of the passwords that they've recently  
used



# Password Complexity

46

- Password cannot equal username
- Allow accounts to be disabled:
  - ▣ User does not use account for a long time
  - ▣ User's password has been comprised.
- Properly store passwords
  - ▣ Don't store in plaintext
  - ▣ Don't encrypt
  - ▣ Use a strong hash
  - ▣ Use a salt
- Multiple rounds of hashing

# Outline



47

1. Access Control Overview
2. Authentication Fundamentals
3. Web Application Authentication
4. Security Password – Based Web Authentication
5. Securing Web Authentication Mechanism

# Securing Web Authentication Mechanism



48

- ❑ Secure the transmission
- ❑ Allow Account Lockout: prevent a user from being able to log in after a certain number of failed login attempts.
- ❑ Allow Account to be disabled
- ❑ No Default Account
- ❑ Don't hard – code credentials
- ❑ Avoid “Stay Sign In”





# QUESTIONS And Answers

